AIGym

System Design Description

Abstract
This document describes the design and implementation of the AIGym support system of Arrowhead 5.2.2 (`ah5-AIGym-java-spring`).

Contents

1 Overview **3**

2 Implementation **4**
2.1 Implementation language and tools 4
2.2 Independent source and build ownership 4
2.3 Functional properties implementation 4
2.4 Non functional properties implementation 4
2.5 Deployment 5
2.6 Local modifications 5
2.7 Verification 5

3 Services **6**

4 References **6**

5 Revision History **8**
5.1 Amendments 8
5.2 Quality Assurance 8

1 Overview

This document describes the AIGym system implementation, which realizes the AIGym SysD [1]. AIGym is an experimentation system for machine learning in the local cloud: it collects training samples, predicts with k nearest neighbours and consumes the `sysMonitorData` stream. In Section 2, we describe implementation details of the system.

2 Implementation

This implementation is based on the SysD document AIGym [1] and is located in `ah5-AIGym-java-spring`.

2.1 Implementation language and tools

- Java 21, Spring Boot (web), Eclipse Paho MQTT, RabbitMQ AMQP client.
- Storage: in memory.

2.2 Independent source and build ownership

The project folder owns its application source, Maven build, tests, executable JAR, Dockerfile, configuration and Compose definition. No source or image is taken from a sibling support folder or JavaSupportRuntime. Build with Java 21 and `mvn -B clean verify`; a populated external Maven dependency cache permits offline builds.

AIGym retains its independent Spring Boot parent and application source. It does not use the Arrowhead common runtime or another support-system artifact.

The packaged artifact is `arrowhead-ai-gym-5.2.2.jar`. The image builds and tests this folder only; the system has a fixed application main class.

2.3 Functional properties implementation

The following components implement the system:

Component	Responsibility
`AiGymController`	HTTP operations.
`ModelStore`	k-NN store.
`AiGymRuntime`	MQTT intake and sysMonitor.
`ArrowheadGateway`	Registration, identity, orchestration, authorization.
`ArrowheadSecurityFilter`, `CrlRevocationConfig`	Caller authentication and CRL.

Every request passes `ArrowheadSecurityFilter` (identity token verified with Authentication and matched to the certificate; ConsumerAuthorization `verify` for the operation) before the controller.

2.3.1 Data handled and stored

Models and observations in memory.

The data models of the interfaces are Java records (of `eu.arrowhead.dto` in the common library, or of the module); the IDD documents name them without the `DTO` suffix. The records used by this system are: `Prediction`, `PredictionRequest`, `Sample`, `SampleRequest`.

2.4 Non functional properties implementation

2.4.1 Security

Spring TLS with mandatory client certificates and `CrlRevocationConfig`; `ArrowheadSecurityFilter` binds the identity token to the certificate.

2.4.2 Power management

None; the system is a server process without power states. The `sysMonitor` `power` observation always reports line supply.

2.4.3 Internal monitoring

Logs go to stdout. The system publishes its `sysMonitor` observations every 30 s (see the sysMonitor IDD).

2.4.4 Configuration

Configuration is read from environment variables (listed in the SysD); secrets are read from files (`*_FILE` variables or mounted credential bundles).

2.5 Deployment

The local `compose.yaml` selects only this support system. It joins an operator-selected external cloud network; it does not start core systems or another support system. Copy `.env.example`, provision this system's own identity and credential files, validate the Compose definition, then run `docker compose up -d -build` from this folder. The image uses the secure external `config/application.properties`, mutual TLS, outsourced authentication, certificate/token binding and a mounted CRL. Secrets are read through Spring's file configuration tree. The app runs as UID 10001 with a read-only root filesystem and bounded non-executable temporary/log storage.

Registry, Authentication, DSO, ConsumerAuthorization and required brokers/providers remain network dependencies. Independent deployment does not remove those service contracts. The listener health check is only a liveness check; inspect successful application startup, registration and authenticated service access before acceptance. The local `DEPLOYMENT.md` supplies the complete procedure. The previously recorded live-cloud deployment is historical baseline evidence; the new packaging does not change a live cloud.

2.6 Local modifications

Notes on this implementation in the workspace:

- Workspace system (5.2.2 naming).

2.7 Verification

Independent folder-only Maven builds run the project's JUnit suite. Artifact inspection checks the system main/source and absence of JavaSupportRuntime or other support-system implementations. Images and project-local Compose definitions are tested with private disposable state and generated test identities. Spring startup and authenticated HTTPS requests use isolated mutual-TLS core-service fixtures; missing-token and missing-certificate requests are refused. The distribution verification report records exact results and limitations. Broker, NTP and inter-cloud acceptance remain separate from deployment/startup validation.

3 Services

The implementation of the services is based on the SD and IDD documents referenced below.

Services produced	**SysD ref**	**SD ref**	**IDD ref**
`aiGym`	[1]	[2]	[3]
`sysMonitor`	[1]	[4]	[5]
Services consumed	**SysD ref**	**SD ref**	**IDD ref**
systemDiscovery, serviceDiscovery	[6]	[7]	–
identity	[8]	[9]	–
authorization	[10]	[11]	–
serviceOrchestration	[12]	[13]	–
sysMonitorData	[14]	[15]	–
sysMonitor (publisher)	AMQP broker (SysMonitor)	–	–

4 References

[1] J. Delsing, “AIGym – System Description,” Eclipse Arrowhead, SysD 5.2.2, 2026, ah5-support-java-spring/AI-gym/documentation/v5.2.2/AIGym-v5.2.2-SysD.pdf.

[2] ——, “aiGym – Service Description (AIGym),” Eclipse Arrowhead, SD 5.2.2, 2026, aIGym-v5.2.2-aiGym-SD.pdf.

[3] ——, “aiGym HTTP/TLS/JSON – Interface Design Description (AIGym),” Eclipse Arrowhead, IDD 5.2.2, 2026, aIGym-v5.2.2-aiGym-HTTP-IDD.pdf.

[4] ——, “sysMonitor – Service Description (AIGym),” Eclipse Arrowhead, SD 5.2.2, 2026, aIGym-v5.2.2-sysMonitor-SD.pdf.

[5] ——, “sysMonitor AMQP/TLS/JSON – Interface Design Description (AIGym),” Eclipse Arrowhead, IDD 5.2.2, 2026, aIGym-v5.2.2-sysMonitor-AMQP-IDD.pdf.

[6] ——, “ServiceRegistry – System Description,” Eclipse Arrowhead, SysD 5.2.2, 2026, ah5-core-java-spring/serviceregistry/documentation/v5.2.2/ServiceRegistry-v5.2.2-SysD.pdf.

[7] ——, “systemDiscovery – Service Description (ServiceRegistry),” Eclipse Arrowhead, SD 5.2.2, 2026, ah5-core-java-spring/serviceregistry/documentation/v5.2.2/ServiceRegistry-v5.2.2-systemDiscovery-SD.pdf.

[8] ——, “Authentication – System Description,” Eclipse Arrowhead, SysD 5.2.2, 2026, ah5-core-java-spring/authentication/documentation/v5.2.2/Authentication-v5.2.2-SysD.pdf.

[9] ——, “identity – Service Description (Authentication),” Eclipse Arrowhead, SD 5.2.2, 2026, ah5-core-java-spring/authentication/documentation/v5.2.2/Authentication-v5.2.2-identity-SD.pdf.

[10] ——, “ConsumerAuthorization – System Description,” Eclipse Arrowhead, SysD 5.2.2, 2026, ah5-core-java-spring/consumer-authorization/documentation/v5.2.2/ConsumerAuthorization-v5.2.2-SysD.pdf.

[11] ——, “authorization – Service Description (ConsumerAuthorization),” Eclipse Arrowhead, SD 5.2.2, 2026, ah5-core-java-spring/consumer-authorization/documentation/v5.2.2/ConsumerAuthorization-v5.2.2-authorization-SD.pdf.

[12] ——, “DynamicServiceOrchestration – System Description,” Eclipse Arrowhead, SysD 5.2.2, 2026, ah5-core-java-spring/serviceorchestration-dynamic/documentation/v5.2.2/DynamicServiceOrchestration-v5.2.2-SysD.pdf.

[13] ——, “serviceOrchestration – Service Description (DynamicServiceOrchestration),” Eclipse Arrowhead, SD 5.2.2, 2026, ah5-core-java-spring/serviceorchestration-dynamic/documentation/v5.2.2/DynamicServiceOrchestration-v5.2.2-serviceOrchestration-SD.pdf.

[14] ——, “SysMonitor – System Description,” Eclipse Arrowhead, SysD 5.2.2, 2026, ah5-support-java-spring/SysMonitor/documentation/v5.2.2/SysMonitor-v5.2.2-SysD.pdf.

[15] ——, “sysMonitorData – Service Description (SysMonitor),” Eclipse Arrowhead, SD 5.2.2, 2026, sysMonitor-v5.2.2-sysMonitorData-SD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing
2	2026-10-25	5.2.2	Self-contained build, private library source, configuration, images, database and isolated deployment verification.	Codex (draft)
3	2026-10-30	5.2.2	Checked against the primary Java source tree (Eclipse-Arrowhead-java): services, operations, paths and data models unchanged.	Claude Opus 5.5 (draft)

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	